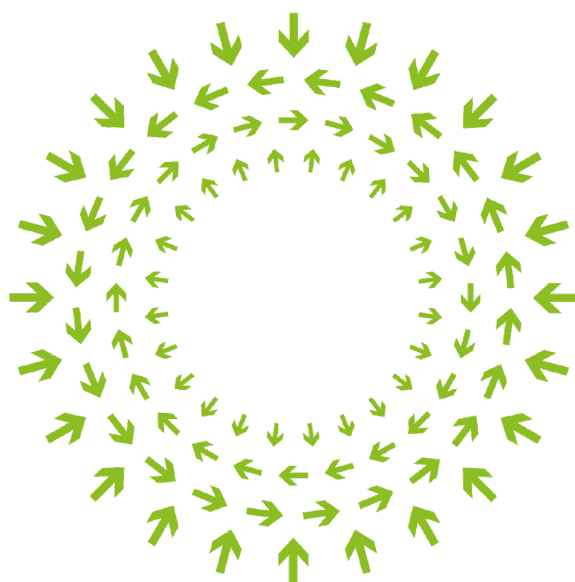




OPEN

Compute Project

RAS API Specification Version 0.95.rc1WIP

Version History

Note, refer to the OCP Contribution Versions, Revisions and Errata best practices documentation. Generally speaking, Versions and revisions are made to this document and logged here. Errata is a separate document such that the contribution specification document it refers to was not revised. Ex: Version 1, Errata E1

Date	Version	Revision	Author	Description
February 2023	0.8	1	Intel Corporation Google LLC	Initial specification released to OCP.
December 2025	0.9	1	AMD, Ampere, Arm, Dell, Google, Intel, Meta, Microsoft, Oracle, PureStorage, Synopsys, Yogesh Varma (independent)	Interim release to reflect change of direction, to give visibility to industry partners. Revised with CPER+CPAD requirements. Removed sections that became inapplicable due to redefinition of the abstraction layer.
June 2026	0.95. rc1 WIP		AMD, Ampere, Arm, Dell, Google, Intel, Meta, Microsoft, Oracle, Everpure, Synopsys	Establishes the foundational normative framework to enable proof-of-concept development (e.g. CPER/CPAD pipelines) and ecosystem enablement (e.g., Redfish, PLDM).

Current Template Version

3 Layer (Base, Design and Product) Specification Template V1.0.0

Effective January 2024

Table of Contents

Version History.....	2
Current Template Version.....	2
1. License.....	4
2. Acknowledgements.....	6
3. Compliance with OCP Tenets.....	7
3.1. Openness.....	7
3.2. Efficiency.....	7
3.3. Impact.....	7
3.4. Sustainability.....	7
4. Scope.....	8
5. Terminology.....	9
6. Requirements.....	9
7. Overview.....	10
7.1. RAS API Relationship to Existing Platform Management Protocols.....	12
8. Base Specifications.....	14
8.1. General guidelines.....	15
8.2. Abstraction Interface.....	15
8.3. Command Primitives.....	16
8.3.1. Discovery.....	17
8.3.1.1. Commands for Discovery.....	17
8.3.2. Configuration.....	17
8.3.2.1. Commands for Configuration.....	18
8.3.3. Error Reporting Queues.....	18
8.3.3.1. Commands for Error Queue Management.....	20
8.3.4. Platform Action Status Queue.....	20
8.3.4.1. Commands for Platform Action Status Queue Management.....	21
8.3.5. Action Queue.....	22
8.3.5.1. Commands for Action Queue Management.....	22
8.3.6. Example Command Sequences.....	22
8.3.6.1. Common Error Analysis and RAS Action Flow.....	23
8.3.6.2. CPAD Rejection by Policy.....	24
8.3.6.3. Error Record Notification.....	25
8.4. Data Structures.....	26
8.4.1. RAS API Coordinate System Overview.....	27
8.4.2. Common Platform Error Record (CPER).....	29
8.4.3. Common Platform Action Descriptor (CPAD).....	30
8.4.3.1. CPAD General Format.....	30

8.4.3.2. CPAD Record Header.....	31
8.4.3.3. CPAD Section Descriptor.....	34
8.4.3.4. CPAD Standard Action ID Values.....	38
8.4.3.5. CPAD Section Body.....	39
8.4.4. Platform Action Event Reporting.....	39
9. Glossary of Terms.....	46
10. References.....	46

1. License

Contributions to this Specification are made under the terms and conditions set forth in **Modified Open Web Foundation Agreement 0.9 (OWFa 0.9)**. (As of October 16, 2024) (“Contribution License”) by:

Alibaba
AMD
Ampere
Arm
ByteDance
Dell
Everpure
Google
Hewlett Packard Enterprise
Insyde
Intel
Inventec
Meta
Microsoft
NVIDIA
Oracle
SKHynix
Synopsys
Tencent

Usage of this Specification is governed by the terms and conditions set forth in **Modified OWFa 0.9 Final Specification Agreement (FSA)** (As of October 16, 2024) (“**Specification License**”).

You can review the applicable Specification License(s) referenced above by the contributors to this Specification on the OCP website at <https://www.opencompute.org/contributions/templates-agreements>.

For actual executed copies of either agreement, please contact OCP directly.

Notes:

The above license does not apply to the Appendix or Appendices. The information in the Appendix or Appendices is for reference only and non-normative in nature.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP "AS IS" AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY DAMAGES WITH RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

2. Acknowledgements

The Contributors of this Specification would like to acknowledge the following for their feedback:

Name of the CLA Participant Company	Names (Position) of the Contributing Individuals Representing Company
AMD	Vilas Sridharan Sarathy Jayakumar
Ampere	Seb Hily
Arm	Dean Liberty Jose Marinho (Co-lead OCP RAS API WS)
ByteDance	Terry Gong
Everpure	Meeta Saggi
Google	Sean Malley
Hewlett Packard Enterprise	Mohan Parthasarathy
Intel	Yogesh Varma
Meta	Anil Agrawal Sukay Luhadia
Microsoft	Carlos Valin Drew Walton (Co-lead OCP RAS API WS) Rama Bhimanadhuni
NVIDIA	Shubhada Pugaonkar
Oracle	Aimer Bhat
Qualcomm	Ashok Raj
Synopsys	Durga Prasad Sahoo

3. Compliance with OCP Tenets

3.1. Openness

The publication of RAS API helps standardize the interfaces for RAS features on the platform, while providing a wide variety of knobs so hardware component suppliers can still innovate and differentiate with RAS features. The main goal of this spec is to facilitate the adoption of RAS features to end customers; making it open while still providing the flexibility of knobs is expected to allow OEMs and CSPs to develop their management systems around this spec.

3.2. Efficiency

Current RAS implementations are proprietary and a great majority of them require firmware expertise to get deployed. The simplification and hardware abstraction that are achieved through RAS API enable smaller teams to experiment and deploy RAS features easily and quickly. This should provide improvements in efficiency for the RAS teams involved.

Also, the standardization of error logs from any type of IP through a common API allows for efforts to be directed towards the analysis and use of the data, rather than in how to extract it, creating yet another efficiency for end customers.

3.3. Impact

Even though the RAS API was developed by Intel and Google, a great amount of the needs and problem statements come from the hardware fault management workstream at OCP. The specification will provide a huge advantage to end customers in their validation and deployment cycles and the ease of use for RAS features.

3.4. Sustainability

A paper by Google [1] analyzed and projected the number of resources wasted due to job failure and relaunch. That study indicated that 12%-20% of compute resources could be saved from current calculations if the right algorithms for predicting job failures were used.

RAS API provides the standardization needed to further those analyses and apply them more broadly to any component in the platform. The savings on compute power will reflect positively not only in energy resources, but on TCO for datacenters.

4. Scope

The scope of this document is to define the RAS API logical abstraction and standardize the use of existing industry-standard management protocols to exchange data between entities conforming to the RAS API. RAS API is an interface for components that defines mechanisms for managing RAS use cases. This document covers the following key aspects:

1. The RAS API abstraction interface.
2. Commands and responses to interact with the RAS API abstraction.
3. Standardized data structures used as payloads.

Additionally, this document may also describe implementation considerations related to particular features.

Out of Scope

The requirements defined in this specification apply when a system implements a particular capability in a manner that conforms with this specification. However, this specification does not specify whether a given system is required to implement that capability.

This specification does not intend to introduce or prohibit a new on-the-wire communication protocol for RAS API. Instead, it defines requirements for RAS use cases that establish a common framework for leveraging existing industry-standard management protocols to achieve consistent RAS behavior across platforms. Such protocols may include, but are not limited to, Redfish, PLDM and other platform-defined management transports.

This specification is not intended to address system-level platform requirements. However, parts of this specification may rely on other relevant specifications or related initiatives. The following references are particularly relevant:

- [OCP Hardware Fault Management Infrastructure Requirements](#)
- [OCP Fleet Memory Fault Management Framework Requirements](#)

5. Terminology

Table 1 – Key RAS API Terms

Term	Description
RAS API	The data exchange and communication protocol for discovery, configuration, and interactions between RAS API agents
RAS API Agent	Entity that understands the RAS API protocol. A RAS API Agent could be either an Endpoint (EP) or an Initiator
RAS API Endpoint	Agent that exposes RAS API abstraction and services the API requests
RAS API Initiator	Agent that initiates the RAS API requests
Transport Layer	The underlying transport and/or data model used to exchange RAS API commands and responses using transport-specific semantics

For additional terminology, please see section 9. Glossary of Terms.

6. Requirements

This section outlines the foundational requirements necessary to achieve the objectives of the RAS API initiative. These requirements are derived from common pain-points and desired enhancements generally accepted within the RAS community. Furthermore, they are consistent with the requirements outlined in the [OCP Hardware Fault Management Infrastructure Requirements](#) document produced by the OCP Hardware Fault Management team.

1. The OCP RAS API specification shall be component and interface agnostic, meaning it can be applied to all components (e.g. CPU, GPU, Accelerators) and usage models (e.g. In-band and/or Out-of-Band).
2. The RAS API abstraction should enable Server Fleet Operators to write data center software code that remains consistent across new hardware generations and suppliers, provided they comply with this specification.
3. The RAS API shall enable dynamic discovery, enumeration and configuration of supported RAS capabilities.

4. The RAS API shall mandate a standardized error record format and queuing system, optimizing error data collection for large-scale operations.
5. The RAS API shall support requests for actions to endpoints in a standardized manner. Actions may be associated with error records.
6. The RAS API definition shall include provisions for supporting proprietary functions in a manner that allows suppliers to innovate without necessitating modifications to Server Fleet Operator infrastructure.
7. The RAS API definition shall include provisions for version control and maintain backwards compatibility with prior versions.
8. The RAS API shall enable configuring, logging errors from, and sending platform actions regarding physical Field Replaceable Units (FRUs).

7. Overview

The RAS API provides an abstraction layer that standardizes fault management interfaces. The RAS API consists of RAS API Endpoints and RAS API Initiators, as shown below. The combination of Initiator and Endpoint form a RAS API instance which handles RAS API requests:

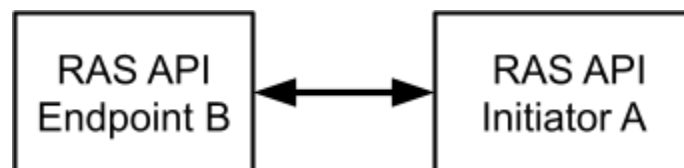


Figure 1: RAS API Instance

The arrow represents an implementation specific transport layer (e.g. PCIe, PLDM, etc.) operating within the RAS API instance and exchanging data using semantics standardized for the RAS API use case. The mapping of the RAS API protocol to the transport layer is outside the scope of this document.

The RAS API networks can be arranged in up to two-level deep hierarchical topologies, as shown in the following figure:

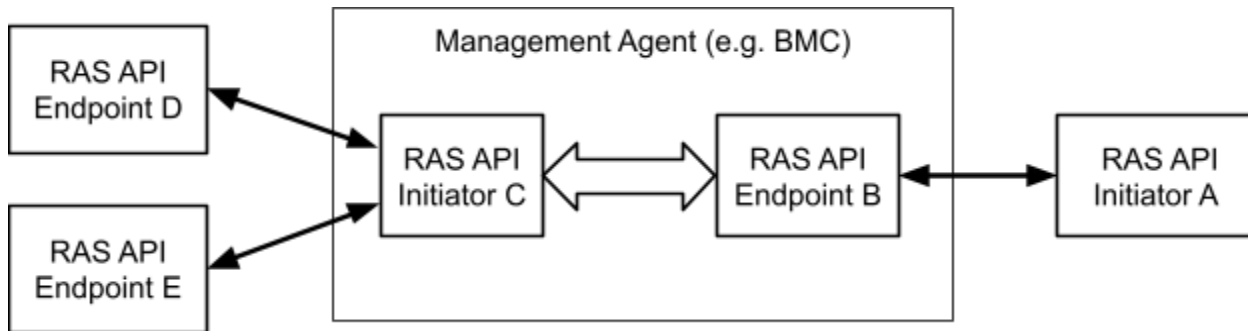


Figure 2: Example Hierarchical RAS API Topology

One key use case for such a two-level deep topology reflects the [OCP Hardware Fault Management Infrastructure Requirements](#) document where endpoints D and E are implemented in silicon such as CPUs and IO devices and where initiator C and Endpoint B may represent an In-Band or Out-of-Band Management Agent. In such a use case Initiator A may act as a collector that feeds error logs to error analysis tools and sends CPADs from the analysis tools back to the RAS API endpoints. The thick arrow between Endpoint B and Initiator C is implementation specific to the management agent and not presently defined in the specification.

The RAS API endpoints provide a standard way for management agents to discover, configure, collect error data from, and perform platform actions on said endpoints.

RAS API Endpoints can be connected in a hierarchical manner as shown above. The arrows between components represent system management transport layers to which the RAS API has been mapped. The RAS API operations themselves are transport layer agnostic. The OCP RAS API workstream will coordinate with the appropriate standards bodies, e.g. DMTF, ACPI, to map RAS API operations to management transport layers.

Errors are collected from RAS API endpoints using the Common Platform Error Record (CPER) format via Initiators that execute RAS API commands. CPER is defined in the [UEFI specification - Appendix N](#). The Initiator triggers actions on RAS API endpoints by sending them Common Platform Action Descriptors (CPADs). A CPAD may also describe manual interventions such as replacing parts.

Note: The CPAD definition is wholly contained in the current release of this specification.

The RAS API also defines how fields in CPERs and CPADs descriptors are used to ensure that CPADs can be delivered to the correct RAS API endpoints. CPER and CPAD section bodies may contain data in a proprietary format and CPAD descriptors support both standard and proprietary actions.

This specification provides a set of minimum requirements for component suppliers and data structures, opcodes, and commands for various use cases. Examples may be provided to clarify

the collective needs related to HW fault management of the data center fleet. These guidelines will encourage component suppliers to lead and refine protocol specifications that fulfill these requirements. Consequently, software drivers developed in accordance with these enhancements will comply with both the RAS requirements and the software interface definitions outlined in this document.

7.1. RAS API Relationship to Existing Platform Management Protocols

Existing management protocols commonly deployed in platform management environments (e.g., Redfish, PLDM) already provide mechanisms for:

- Transferring data between components (endpoints) and management controllers (initiators),
- Discovering and exchanging configuration data,
- Supporting asynchronous event notifications, and
- Communicating structured management data.

These capabilities are sufficient to support the RAS use cases addressed by this specification **when used in a standardized manner**. Accordingly, this specification does not introduce new packet framing formats, transport-layer message headers or trailers, protocol-specific command opcodes, or completion or response code definitions for RAS API operations.

Instead, the RAS API defines the functional expectations associated with operations such as retrieval of CPERs or delivery of CPADs. Protocol-specific mappings translate these operations into the appropriate commands or data exchanges supported by the underlying transport. The standards bodies that own the platform management protocols will own mapping the RAS API to management protocol.

The following drawing illustrates an example of this protocol mapping, showing how Redfish and PLDM might be used to send RAS API commands and responses.

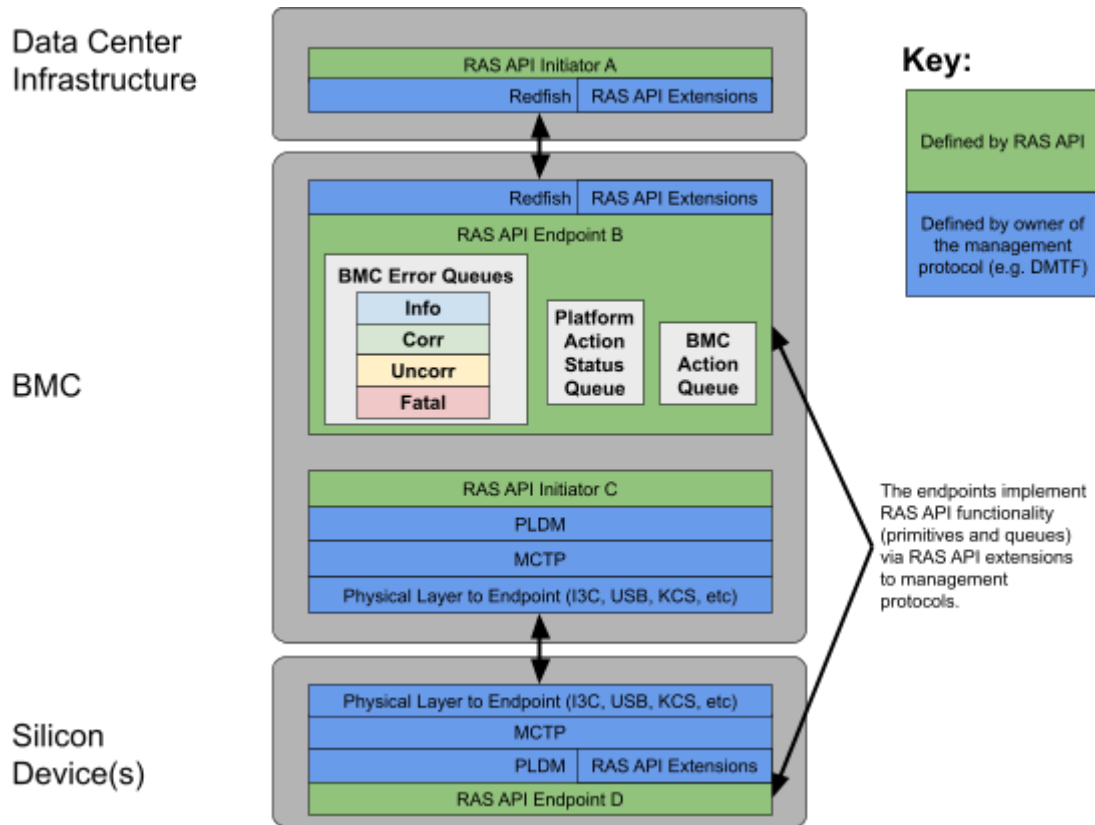


Figure 3: Example RAS API Protocol Mapping

Following is an illustration of RAS API in-band flow:

The following diagram gives an example for how the RAS API might be standardized for PLDM or any other server management protocol.

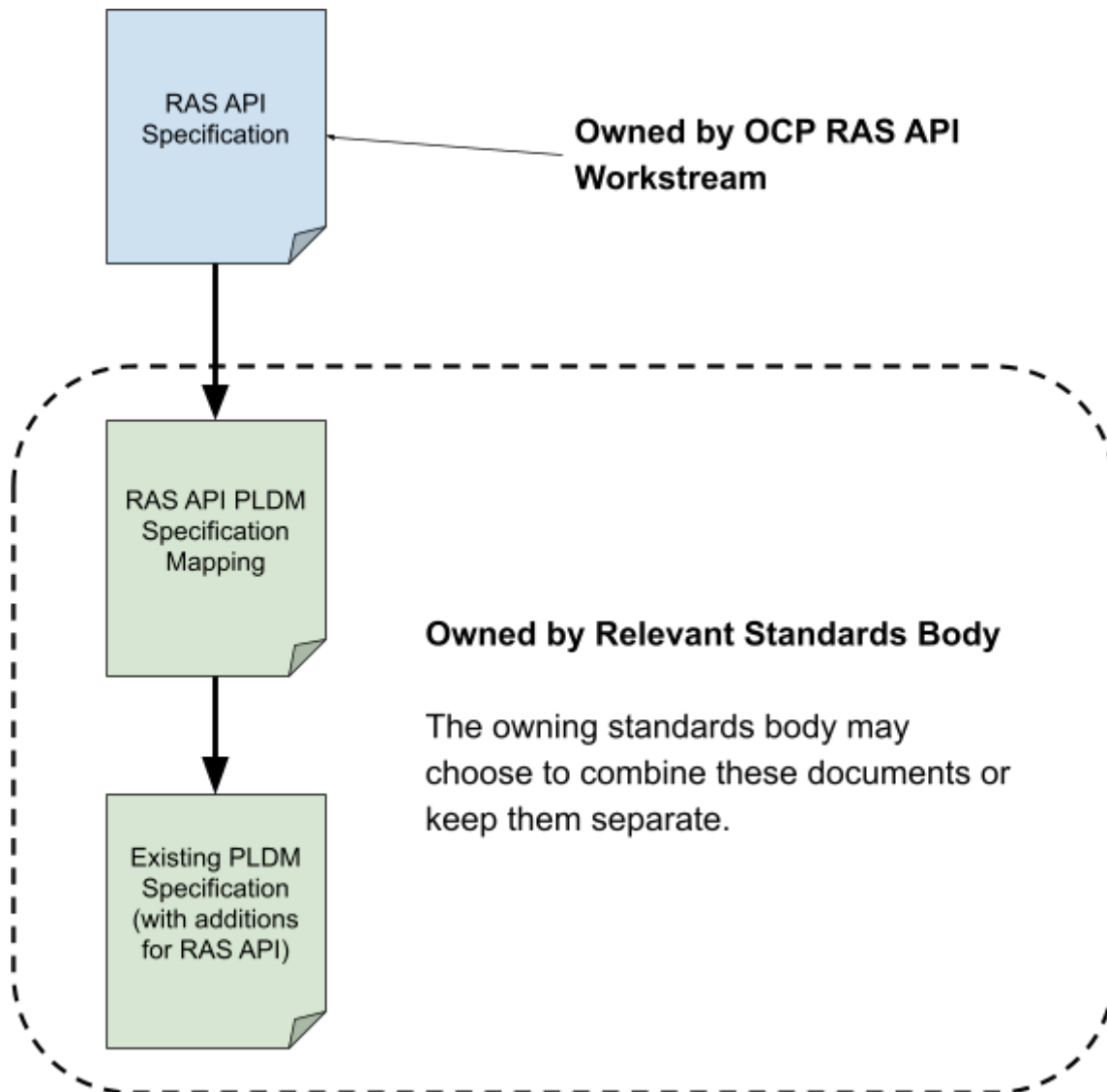


Figure 4: RAS API Documentation for Management Protocols

8. Base Specifications

This section contains the normative requirements of the OCP RAS API specification. All statements and requirements described herein use normative language (such as "shall," "must," and "should") to define the mandatory behaviors, interfaces, and data structures that must be implemented for compliance. Implementers are expected to adhere to these requirements to ensure interoperability and conformance with this specification.

8.1. General guidelines

To ensure the broadest impact and facilitate a unified approach to RAS features within the data center ecosystem, it is critical that all specifications, including this OCP RAS specification, are shared openly. This document is being crafted with transparency in the open-source domain, and it is imperative that any companion specifications required for a comprehensive solution follow suit. By contributing these additional protocol layer specifications to open-source projects, we can extend the benefits beyond a proprietary framework, allowing for widespread adoption and collaboration across multiple component suppliers.

8.2. Abstraction Interface

RAS API Endpoints shall expose an abstraction interface comprising the following components:

- Discovery
- Configuration
- Independent sets of logical error and event reporting queues per supported interface instance
- Independent logical action queues per supported interface instance

The following diagram illustrates this concept.

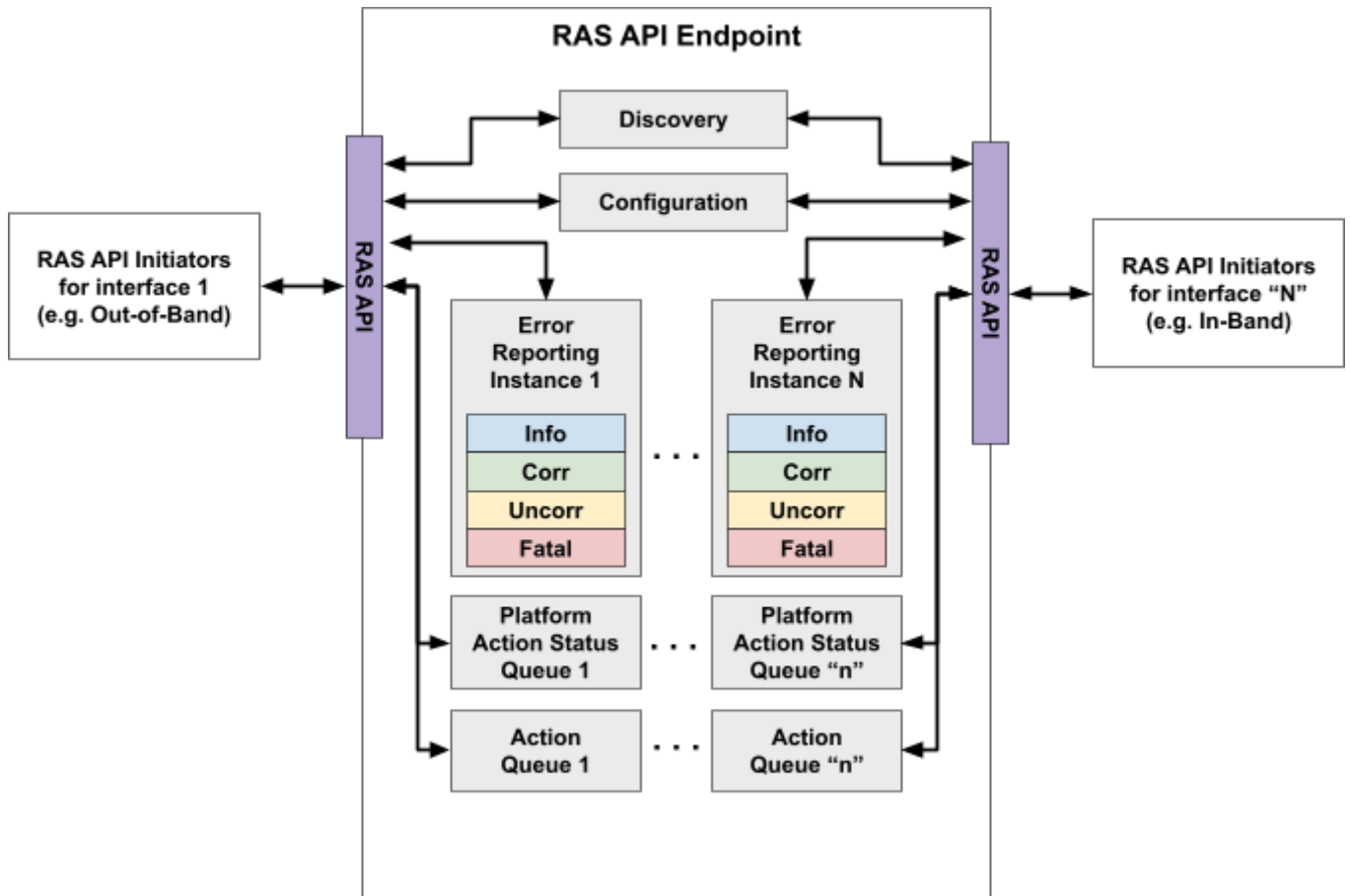


Figure 5: High-level diagram of RAS API Endpoint abstraction

The **key use case** targeted by this specification is to expose two sets of separate logical error/event reporting and action queues, intended for dedicated access and management by out-of-band and in-band RAS API initiators (i.e. “n” = 2). However, the abstraction requirements are specified to accommodate potential use cases that may necessitate more than two sets of logical queues.

8.3. Command Primitives

To support mapping the RAS API to platform management protocols, this specification describes command primitives and the essential behavior of those primitives.

RAS API operations are expressed in this specification using abstract semantic functions, such as retrieval of error records from a specified logical queue, or delivery of platform action requests to a

target endpoint. These abstract operations define the expected behavior of a compliant endpoint from the perspective of a RAS API initiator.

A RAS API initiator may realize these operations by translating the abstract semantics into requests using a supported management protocol. For example, a request to retrieve error records may be translated into a Redfish REST operation or a PLDM command.

The endpoint processes these protocol-native requests using its existing functionality and returns data that conforms to the expectations defined by this specification. The "protocol-native" requests are translated or mapped back to RAS API requests to be processed, at least conceptually. The RAS API specifies functions, not PLDM or Redfish functions. For example, returning CPERs of a requested severity with a "200 OK" response for Redfish while PLDM might use 0x0 for success.

The RAS API primitives are not transmitted on the wire as a distinct protocol. Rather, they are realized through standardized usage of existing protocol capabilities. These primitives are expected to be used to construct interfaces appropriate to whatever management protocol is being used. This specification will list minimum parameters and responses for the RAS API primitives, but allows the protocol mapping to specify the details in a manner appropriate to the protocol.

The following sections describe the RAS API primitives.

8.3.1. Discovery

The RAS API Endpoint shall provide functionality to enable programmatic discovery of RAS API support level and the capabilities supported by the Endpoint.

8.3.1.1. Commands for Discovery

The RAS API supports discovery commands that allow an initiator to discover endpoints, including their type and identity. The discovery commands provide information necessary for a server operator's fault management infrastructure to configure endpoints, collect error data from the endpoints, analyze that data, and send platform actions back to the endpoint. These discovery commands will be described in the next version of the specification.

8.3.2. Configuration

The RAS API Endpoint shall provide functionality to configure the endpoint. More details will be added in a future version of the specification.

8.3.2.1. Commands for Configuration

The following table summarizes the functionality enabled by the command set for RAS API configuration capabilities.

Table 2 – Configuration Commands

Function	Min Response	Command
Retrieve basic information about each RAS API EP's capabilities	Config data	This will be defined in a future revision
Enable RAS API initiators to configure EP's settings.	Completion code	

More details, including potential conditional and optional functionality, will be provided in future releases of this specification.

8.3.3. Error Reporting Queues

The specification mandates separate sets of queues for error reporting to each RAS API interface to prevent any interaction between them. Each set contains four severity-based queues. Error records are categorized into the queue that matches its severity.

Errors reported on these queues shall follow the CPER format, as detailed in [Common Platform Error Record \(CPER\)](#). RAS API Endpoints shall use the severity field in the CPER header (corresponding to the most severe error section in the CPER) to assign error records to their corresponding error queue.

The error queues are defined according to CPER severities as follows:

- **Fatal Error Queue:** For reporting errors containing critical events that prevent continued system operation or fatal issues (e.g. system crashes).
- **Recoverable Error Queue (also called Non-Fatal Uncorrected):** For reporting uncorrected errors that require immediate management-entity intervention, often part of recovery flows (e.g., poison-based containment).
- **Corrected Error Queue:** For reporting errors that have been corrected by hardware and do not require immediate action but may be useful for Predictive Failure Analysis (PFA) or preemptive policies (e.g., corrected errors).
- **Informational Error Queue:** For reporting errors that may require future actions, such as poison generation.

The following additional requirements shall apply to all error queues:

- Queue capacity is implementation specific and should be sized according to expected error rates and error log sizes for each severity class. For instance, corrected and informational queues may require greater capacity than recoverable and fatal queues.
- Each queue shall implement First-In, First-Out (FIFO) behavior.
- Each queue shall support configuration of its interrupt policy. For each queue, interrupt policy configuration shall be independent of other queues. This allows RAS API Initiators to set the most suitable mode of operation (e.g., polling vs interrupt-driven) in real time.
- Error records should be persistent across a non-power cycle platform reset. If an error record is lost due to such a reset, the component shall signal such a loss.
- Queues may optionally provide configurable interrupt thresholds (e.g., first error, half-full, 90% full) or rate-limiting features to aid in the management of error-storm scenarios.
- The endpoint shall indicate an overflow when queues lose or drop entries.

NOTE: The RAS API expects deferred errors like poison generation to be treated as informational CPERs and placed in the informational queue.

[OCP Hardware Fault Management \(HWFM\) Infrastructure Requirements](#) map various server platform error types to UEFI CPER defined error severities in the section titled “Hardware Component Error Classification.”

8.3.3.1. Commands for Error Queue Management

This section outlines a set of RAS API error queue management operations that enable initiators to retrieve and clear error queues.

Table 3 – Commands for Error Queue Management

Function	Parameter	Expected Response	Details
Get CPER	The Error Queue to be read	- Returns CPER - Indication of whether the queue overflowed - Success or failure code	Gets one or optionally more CPERs
Clear CPER	- The Error Queue to be cleared - The Record ID of the CPER to be cleared	Success or failure code	Clears the head of the error queue.

Protocol mappings for reading the Error Queues are expected to handle, in a protocol-specific manner, cases where CPERs are too large for a single protocol transaction.

8.3.4. Platform Action Status Queue

The RAS API Endpoint shall expose a logical queue used for RAS-related events that are not categorized as errors, including Platform Action Events. The maximum depth of this queue is implementation specific.

The key use case is to report results of platform actions requested through the Action Queue (i.e. CPADs). Passing the result of a RAS action back to the initiator allows analysis tools to determine whether the Endpoint applied the action successfully and gauge the effectiveness of a successfully applied action. This prevents analysis tools from basing recommendations on actions that were assumed complete but were not, and avoids repeatedly suggesting ineffective actions (e.g., escalating to part replacement when a self-repair action failed repeatedly).

Events reported on this queue shall use the CPER format and are intended to be collected by the same Initiator that retrieves CPERs from the other error queues. The CPER definition for these events is provided in the section titled [Platform Action Event Reporting](#).

The following additional requirements shall apply to Platform Actions Status queues:

- Queue capacity is implementation specific and should be sized according to expected event rates.
- This queue shall implement First-In, First-Out (FIFO) behavior.
- This queue shall support independent configuration of its interrupt policy. This allows RAS API Initiators to set the most suitable mode of operation (e.g., polling vs interrupt-driven) at run time.
- This queue shall persist event records across platform resets that do not cycle power.
- This queue may optionally provide configurable interrupt thresholds (e.g., first error, half-full, 90% full).
- The endpoint shall indicate an overflow when queues lose or drop entries.

8.3.4.1. Commands for Platform Action Status Queue Management

This section outlines a set of RAS API Platform Actions Status queue management operations that enable initiators to retrieve, and clear Platform Action Status queues.

Table 4 – Commands for Platform Action Status Queue Management

Function	Parameter	Expected Response	Details
Get CPER	• The Platform Action Status Queue to be read	• Returns CPER • Indication of whether the queue overflowed • Success or failure code	Gets one or optionally more CPERs
Clear CPER	• The Platform Action Status Queue to be cleared • The Record ID of the CPER to be cleared	• Success or failure code	Clears the head of the Platform Action Status queue.

Protocol mappings for reading the Platform Action Status Queue are expected to handle, in a protocol-specific manner, cases where CPERs are too large for a single protocol transaction.

8.3.5. Action Queue

The RAS API Endpoint shall expose a logical queue for receiving requests to execute platform actions. The minimum supported queue depth shall be 1; the maximum depth is implementation specific.

Each action request shall use the Common Platform Action Descriptor (CPAD) format defined in Section [8.3.2](#).

8.3.5.1. Commands for Action Queue Management

The following table summarizes the functionality that must be enabled by the command set to allow RAS API Initiators to interact with each Endpoint's Action queue.

Table 5 – Commands for Action Queue Management

Function	Parameter	Expected Response	Details
Put CPAD	The CPAD	Success or failure code, including a failure code indicating that the command failed because the queue was full.	Used to send CPAD to an endpoint.

A success response from putting a CPAD means that the endpoint received the CPAD. The actual responses from the actions in the CPAD are returned in the Platform Action Status queue as [Platform Action Event](#) CPERs.

Some actions may require an independent event before they can be taken. A common example is a RAS action that takes place on the next boot of the system. RAS API endpoints must be able to hold pending actions until preconditions for the action have been met.

- Queue capacity is endpoint-implementation specific.
- This queue shall implement First-In, First-Out (FIFO) behavior.

8.3.6. Example Command Sequences

This section illustrates example command sequences showing how the RAS API uses CPERs (error logs and Platform Action Events) and CPADs to gather, analyze, and act on hardware error data. These flows show only essential parts of the RAS API flow and omit details of the protocol

mappings and omit details of the Server Fleet Operator's infrastructure. This section is non-normative.

8.3.6.1.Common Error Analysis and RAS Action Flow

One of the most common flows is shown below. In this example, the analyzer does not specify an action for the first CPER. This will be a common case. Analyzers may need multiple CPERs to recognize faults and may need to filter out soft errors for which there is no repair. This example shows a simplified use of the Get CPER command. Server management protocols such as Redfish support eventing and may permit the Collection Infrastructure to register to receive CPER events from the RAS API Endpoint.

When the analyzer has enough CPERs to specify a RAS action, it generates a CPAD. The Server Fleet Operator evaluates the CPAD against its policies. If the Server Fleet Operator policy allows the RAS action, it forwards the action to the RAS API endpoint.

The RAS API endpoint gets the CPAD and optionally validates it. The validation might include checking that the CPAD was targeted for this endpoint and that the action(s) suggested in the CPAD are valid. If the CPAD cannot be acted upon immediately, the endpoint might send a Platform Action Event indicating that the action(s) are pending so that the analyzer can see that the action is pending. This is useful in the event that the endpoint sends additional, related CPERs to the analyzer. Since the analyzer can see the pending action(s), it can avoid resending the actions for the new, related errors.

When the preconditions for the RAS Action are met, the RAS API Endpoint emits a CPER containing a Platform Action Event CPER in its Platform Action Status Queue. The Platform Action Event CPER contains the status of the action, exposing it to the analyzer. This allows the analyzer to propose new actions in the event that a CPAD was not successful.

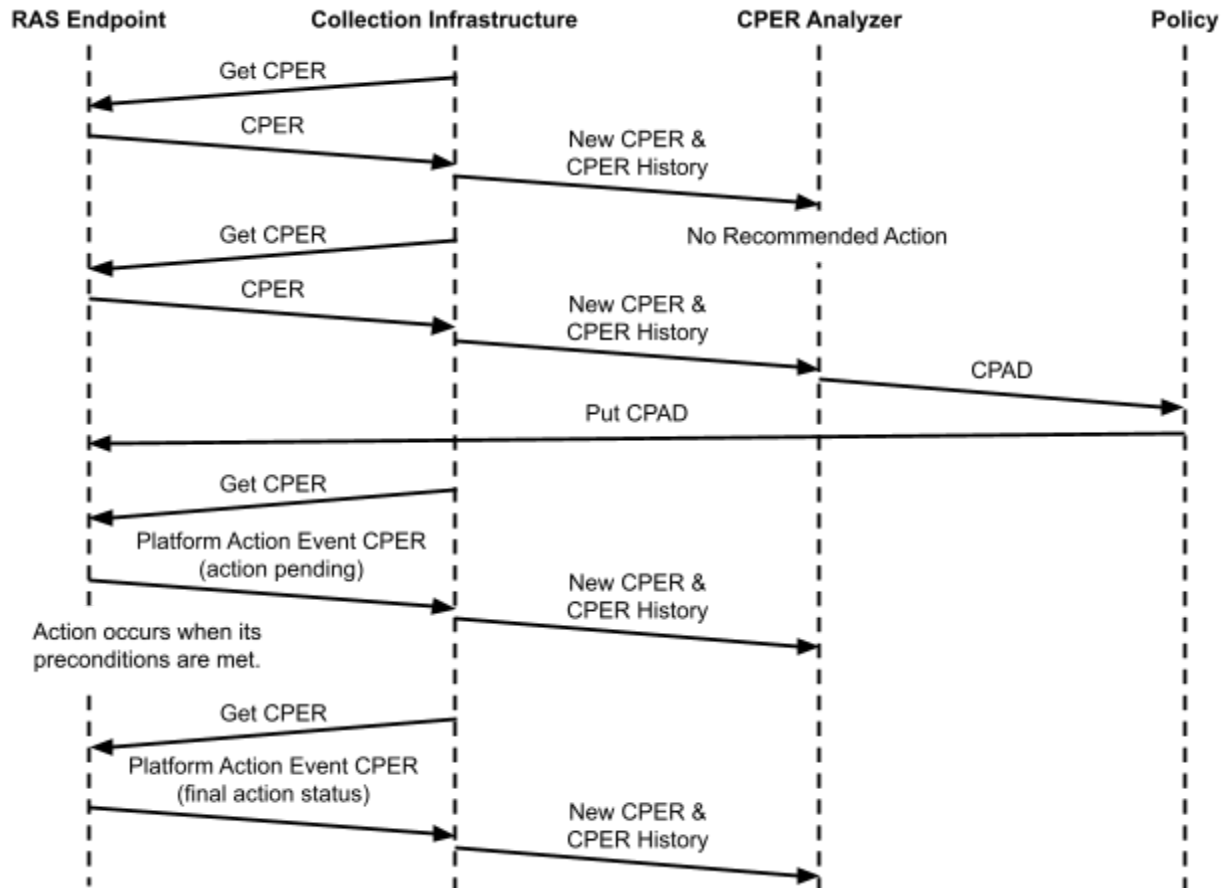


Figure 6: Common Analysis and Action Flow Omitting Details of Management Protocol

8.3.6.2. CPAD Rejection by Policy

The following sequence diagram illustrates how server fleet operators might apply their own policies to the output of CPER analyzers. In this example, the CPER Analyzer emits a CPAD suggesting one or more actions. The server fleet operator's policy engine can use the Creator ID and Action ID along with any other relevant information to decide if the CPAD section should be allowed to proceed.

In this example, the policy engine rejects the CPAD. The policy engine might decide to trigger a part replacement using the PlatformID and FruText fields in the CPAD. The policy engine also creates a Platform Action CPER that contains a Platform Action Event Action Status of "0x0D-Policy rejected this CPAD." This Platform Action Event is created using the fields in the rejected CPAD and is generated in a generalized manner that works for any rejected CPAD. The Platform Action Event is sent to the collection infrastructure which then passes it to the CPER Analyzer. The CPER Analyzer might suggest an alternate action for the problem in another CPAD.

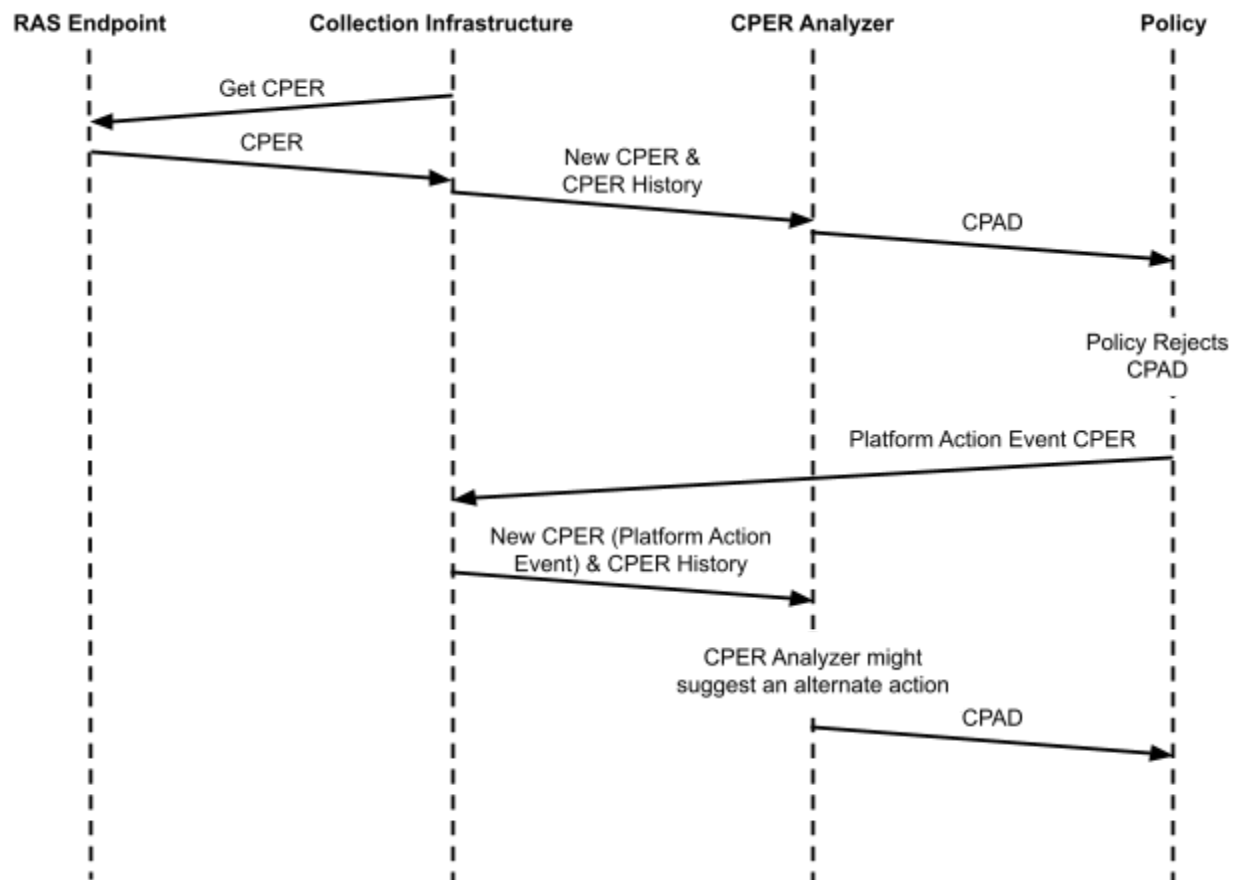


Figure 7: CPAD Policy Rejection

8.3.6.3. Error Record Notification

CPER records are generated asynchronously by a RAS API EP. The transport layer (e.g. PLDM, Redfish) may support asynchronous notifications. The details of the event notification are transport-layer-specific and are outside the scope of this document.

If the transport layer supports event notification, and the EP enables those, the Initiator can register to receive notifications when a new CPER record is available at the EP, as shown in Figure 6 below. The notification registration may allow masking per severity. The CPER notification sent may describe the severity associated. This capability depends on the transport layer eventing capabilities and descriptiveness.

If the transport layer does not support event notifications, then the Initiator must poll for new events. The maximum or minimum required polling frequency is not constrained by this document.

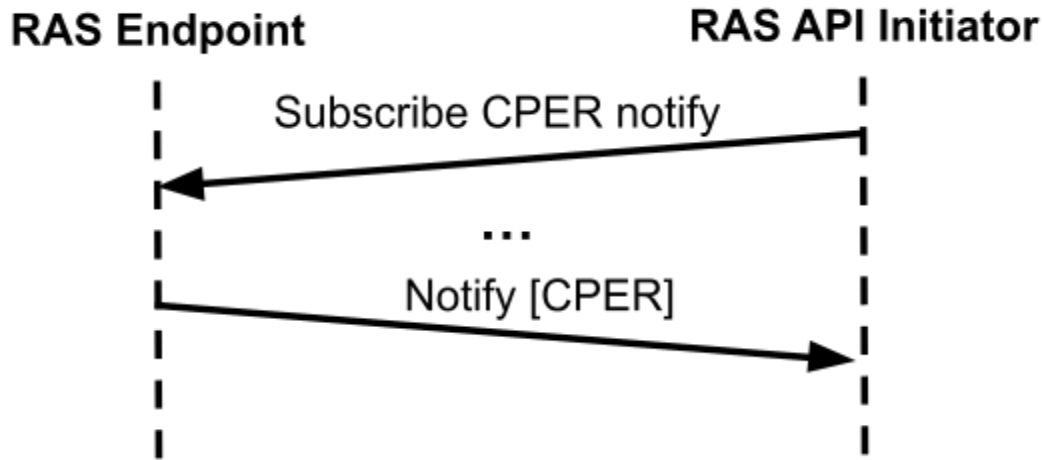


Figure 8: Event driven record retrieval

8.4.Data Structures

This section defines the data structures standardized for use as RAS payloads by this specification. The following table summarizes it.

Table 6 – Data Structures

Category	Data Structure Used
Error Reporting Queue record format	Common Platform Error Record (CPER)
Action Queue record format	Common Platform Action Descriptor (CPAD)
Platform Action Status Queue record format	Common Platform Error Record (CPER), specifically Platform Action Events defined in the section titled “Platform Action Event Reporting,” below.

8.4.1. RAS API Coordinate System Overview

The RAS API standardizes the flow of error data from silicon components to tools that process the error data (analyzers) and standardizes the format for RAS actions to be sent back to the data center and to the silicon components.

To enable this flow, the RAS API standardizes the use of specific fields within CPER and CPAD records to enable end-to-end routing of these data structures to the appropriate destination—for example, CPERs to the correct component supplier analyzer for decoding and analysis, and CPADs to the appropriate RAS API Endpoint or data-center infrastructure for execution. The standardized fields are:

- Header.PlatformID: Uniquely identifies the platform within the fleet.
- Header.PartitionID: Uniquely identifies the RAS API Endpoint (i.e. silicon component) within the platform.
- Header.CreatorID: Identifies the component supplier analyzer responsible for consuming or generating the data structure.
- SectionDescriptor.FRUID: Uniquely identifies the specific Field Replaceable Unit (FRU) instance associated with the error or action, in a manner similar to a serial number.
- SectionDescriptor.FRUText: Uniquely identifies the physical location of the referenced FRU within the platform via visual aids (e.g., silkscreen, labels) for human remediation.

Analyzers are expected to process CPERs and generate CPADs when they identify an action that might mitigate the condition that caused the error(s). The analyzer uses the fields in the CPER(s) mentioned above to create a CPAD that can be routed back to the source of the CPERs.

Each CPAD section represents an action to be performed and a CPAD must have one or more sections. The CPAD section descriptor specifies the action to be taken in the descriptor's ActionID. Some CPADs represent an action that is taken in silicon components such as invoking a RAS feature or driving a self-repair. Other actions specified in CPADs may need to be performed by the equipment operator, e.g. cycle power or replace a Field Replaceable Unit (FRU). Server Fleet Operator Policies control how the data center infrastructure responds to CPADs and can:

- Forward the CPAD to the silicon component that needs to take action
- Reject the suggested action(s)
- Replace the FRU identified in the FRUText
- Or any other action it deems appropriate

The following diagram illustrates an example mapping of these fields to the corresponding platform components involved in RAS API use cases.

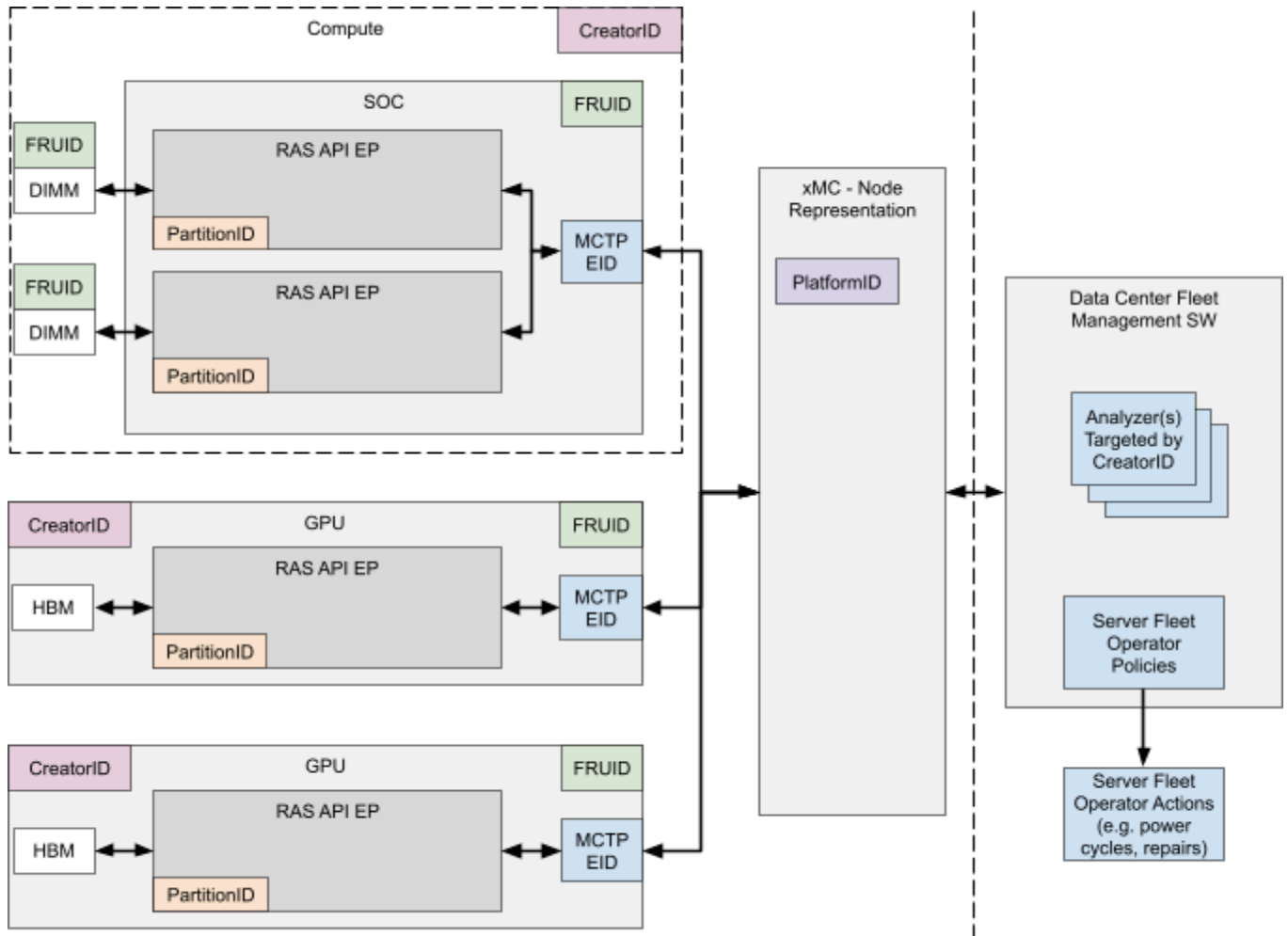


Figure 9: Example of RAS API IDs mapped to various components

In most implementations of the RAS API, each silicon component will have its own RAS API endpoint. In some cases, such as the SoC in the drawing above, silicon may be partitioned such that each partition runs independently, e.g. separate OS instances. Such partitioned silicon components might have more than one RAS API endpoint.

8.4.2.Common Platform Error Record (CPER)

This specification mandates error records to adhere to the Common Platform Error Record (CPER) format, as defined in the [UEFI specification - Appendix N](#).

In addition to complying with the UEFI specification, RAS API Endpoints must populate CPER fields with values enabling the end-to-end flows and use cases defined in [OCP Hardware Fault Management Infrastructure Requirements](#):

1. Header.timestamp field shall contain the local time in BCD format. This allows relative ordering of error records for time-series analysis and correlation with other data from the Endpoint. Refer to the CPER format definition for additional field details.
2. Header.PlatformID field shall contain a platform-assigned GUID that uniquely identifies the platform within the fleet. How this value is generated is outside the scope of this specification.
 - a. The PlatformID value shall enable tracking of platform-level error history across the fleet.
 - b. All CPERs generated by RAS API Endpoints within a platform shall use the same PlatformID value.
3. Header.PartitionID field shall contain a GUID that uniquely identifies the RAS API Endpoint within the platform. This field, along with PlatformID, uniquely identifies the RAS API Endpoint that created the CPER.
 - a. The PartitionID value shall be consistent across all RAS API interface instances (e.g., In-band and Out-of-Band)
4. Header.CreatorID field shall contain a component-supplier-assigned GUID that uniquely identifies the type of device that generated the CPER.
 - a. The CreatorID value shall be used to match CPERs to the corresponding component supplier analyzer.
 - b. The CreatorID value shall be globally unique across component suppliers; component suppliers must publish their assigned CreatorID values to prevent conflicts. Existing industry-standard identifiers (e.g., PCIe vendor/device IDs, IANA numbers) may be leveraged to generate this GUID.
 - c. Header.CreatorID: identifies the component under diagnosis. It can be used for matching the CPER to a first level diagnosis analyzer which may interrogate further details of the CPER and route the CPER to more specialized analyzers. The Server Fleet Operator infrastructure must be able to match the CreatorID to the correct first level analyzer.
5. SectionDescriptor.FruID shall contain a GUID derived from a component-supplier-assigned identifier (e.g. serial number, x86 PPIN) that uniquely identifies the specific Field Replaceable Unit (FRU) instance that experienced the error.

6. SectionDescriptor.FRUText field shall contain a platform-assigned, human-readable ASCII string that allows data center technicians to identify the physical location of the faulty FRU via visual aids in the platform (e.g., silkscreen, labels).
 - a. The FRUText values shall be unique for each FRU within the platform.

Implementation notes:

Refer to Appendix D for more details about key use cases.

In cases where a RAS API endpoint consists of multiple IPs, possibly from different companies, e.g. a CPU core, memory controller, IO interface, etc, the endpoint must have a single CreatorID and an associated analyzer for each of the IPs in the SoC. The routing of CPERs and CPADs inside such an SoC is implementation specific, though IP providers are encouraged to implement standards in CPER and CPAD section bodies to facilitate the implementation of analyzers. Suppliers are encouraged to create RAS API endpoints that cover the installed component, including things like voltage regulators and PCIe retimers.

8.4.3.Common Platform Action Descriptor (CPAD)

This section describes the Common Platform Action Descriptor format for representing platform actions to be performed by RAS API endpoints.

8.4.3.1. CPAD General Format

The general format of the CPAD is illustrated below. The fields in CPAD have the same meaning as fields defined in CPER format in the UEFI specification unless mentioned otherwise. The record consists of a header; followed by one or more section descriptors; and for each section descriptor, an associated section body which describes an action to be taken. Only one action will be contained in each section descriptor and any associated parameters for the action will be contained in the corresponding section body.

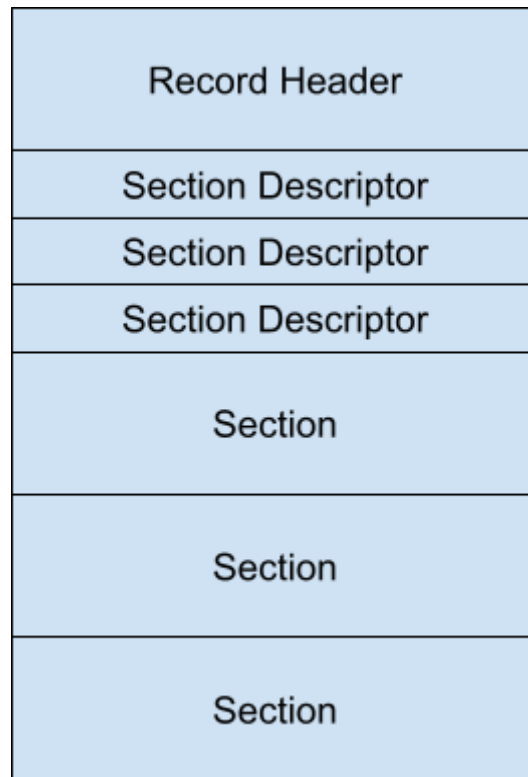


Figure 10: General Format of a Common Platform Action Descriptor (CPAD)

8.4.3.2. CPAD Record Header

The record header of the CPAD includes information which uniquely identifies a set of actions on a given system. The contents of the record header are described in the table below.

The following mandatory fields need to be used consistently for routing by fault management infrastructure. These prevent mistakes when end users direct CPADs to the wrong host.

- Platform ID
- Partition ID
- Creator ID

The Timestamp is also mandatory.

Table 7 – CPAD Record Header

Mnemonic	Byte Offset	Byte Length	Description
Signature Start	0	4	ASCII 4-character array "CPAD" (0x43,0x50,0x41,0x44). Identifies this structure as Common Platform Action Descriptor
Revision	4	2	Consistent with CPER definition (Refer to UEFI spec Appendix N. For this release: • Major: 0x01 • Minor: 0x00
Signature End	6	4	Consistent with CPER definition (Refer to UEFI spec Appendix N
Section Count	10	2	This field indicates the number (N) of valid CPAD sections associated with the record, corresponding to each of the following section descriptors.
Urgency	12	1	This field indicates the highest urgency of the CPAD section descriptors. 0: Not Urgent 1: Urgent 2-255: Reserved values This field is intended primarily for Server Fleet Operator use and is not expected to affect endpoint behavior.
Reserved	13	1	Reserved
Reserved	14	2	Reserved
Validation Bits	16	4	This field indicates the validity of the following fields: • Bit 0 – If 1, the PlatformID field contains valid information • Bit 1 – If 1, the TimeStamp field contains valid information • Bit 2 – If 1, the PartitionID field contains valid information • Bits 3-31: Reserved, must be zero.

Mnemonic	Byte Offset	Byte Length	Description
Record Length	20	4	Indicates the total size of the CPAD in bytes including the size of the record header, all section descriptors, and section bodies.
Timestamp	24	8	The timestamp correlates to the time when the CPAD was created. The format is consistent with the CPER definition (refer to UEFI spec Appendix N)
Platform ID	32	16	Platform-assigned GUID that uniquely identifies the platform.
Partition ID	48	16	A GUID that uniquely identifies the RAS API Endpoint within the platform.
Creator ID	64	16	A component-supplier assigned GUID that uniquely identifies the entity (e.g., Analyzer) that generated this CPAD
Notification Type	80	16	Reserved. Set to zero.
Record ID	96	8	Unique record ID
Flags	104	4	Inherited from the CPER specification, Table N.2. Treated as a bitfield. Bit 2: HW_ERROR_FLAGS_SIMULATED: Qualifies that a CPAD was generated by an error condition that was intentionally caused. This value is set by the analyzer in cases where CPERs contributing to this CPAD have this flag set and cause the analyzer to generate a CPAD. All other values are reserved, the default value is zero.

Mnemonic	Byte Offset	Byte Length	Description
Reserved	108	20	Reserved. Set to zero.
Section Descriptor	128	Nx72	An array of Section Count descriptors for the associated sections. The number of valid sections is equivalent to the Section Count. See section CPAD Section Descriptor for format.

8.4.3.3. CPAD Section Descriptor

The CPAD Section Descriptor is defined in the table below. The 4-byte Severity field in the CPER Section Descriptor definition is replaced with two fields of 1-bytes each, namely Urgency and Confidence. The urgency and confidence fields provide guidance to Server Fleet Operator policies. The intention of the urgency field is to indicate to the endpoint if the root cause of the problem is likely to lead to a service interruption in the near future. The confidence field gives Server Fleet Operators the ability to set a confidence threshold for allowing CPAD actions.

The following fields need to be consistent for routing by fault management infrastructure. These are required to handle the case where a part was replaced before the CPAD was sent/acted upon:

- FRU ID
- FRU Text

Table 8 – CPAD Section Descriptor

Mnemonic	ByteOffset	ByteLength	Description
Section Offset	0	4	Offset in bytes of the section body from the base of the record header
Section Length	4	4	The length in bytes of the section body
Section Revision	8	2	Same definition as CPER.

Mnemonic	ByteOffset	ByteLength	Description
Validation Bits	10	1	This field indicates validity for the following fields: Bit 0 - If 1, the FRUID field contains valid information. Bit 1 - If 1, the FRU text field contains valid information Bit 2 - If 1, the Urgency field is valid. Bit 3 - If 1, the Confidence Field is valid. Bits 7:4 – Reserved, must be zero.
Reserved	11	1	Reserved, must be zero
Flags	12	4	Reserved
Section Type	16	16	This field holds a GUID value indicating the format of the section body. The Section Type is independent of the Action ID. The section types are vendor-defined.
FRU ID	32	16	GUID derived from a component supplier-assigned ID (e.g. serial number) that uniquely identifies the instance of the Field Replaceable Unit (FRU) that had the error
Urgency	48	1	This field indicates a component supplier's recommendation for prompt attention by the Server Fleet Operator, typically due to an elevated risk of service interruption, undesired system behavior, or data corruption. 0: Not Urgent 1: Urgent 2-255: Reserved Values

Mnemonic	ByteOffset	ByteLength	Description
			This field is intended primarily for Server Fleet Operator use and is not expected to affect endpoint behavior.
Confidence	49	1	This field indicates the component supplier's confidence level of the proposed action. 0-100% Confidence level Values of 101-255 are Reserved This field is intended primarily for Server Fleet Operator use and is not expected to affect endpoint behavior.
Reserved	50	2	Reserved
FRU text	52	20	A platform-assigned, human-readable, ASCII string that allows data center technicians to quickly identify the faulty Field Replaceable Unit's (FRU) physical location via visual aids in the platform. (e.g., silkscreen, label). The value is unique in the platform.

Mnemonic	ByteOffset	ByteLength	Description
Action ID	72	2	The Action ID describes the action to be taken. Placing this in the section descriptor allows the action to be identified without having to parse the section body, which may be proprietary. There are two general categories of Action IDs: Standard Actions and Proprietary Actions. Action IDs from 0x0000 to 0x7FFF are reserved for Standard Actions. Action IDs 0x8000 to 0xFFFF are allocated for Proprietary Actions. Standard Actions are listed in the next section of the document. Proprietary Actions are specific to a particular CreatorID and have a consistent meaning for the particular CreatorID. The Action ID is independent of the Section Type.

8.4.3.4. CPAD Standard Action ID Values

Action ID values from 0x0000 to 0x7FFF are reserved for standard actions. These standard actions are defined in the following table.

Table 8 – CPAD Standard Action ID Values

Action ID	Action
0x0000	Do Nothing (No Operation) - Can be used to test CPAD routing - Endpoints respond with Platform Action Event indicating they did nothing.
0x0001	Platform reset without power cycle
0x0002	Cycle platform power
0x0003	Reseat part
0x0004	Shuffle part This might only be used for diagnostic testing since it implies testing after moving the parts
0x0005	Replace part Part to be replaced is specified in FRU text field
0x0006	Inject error
0x0007 to 0x7FFF	Reserved
0x8000 to 0xFFFF	Proprietary actions specific to a particular CreatorID.

8.4.3.5. CPAD Section Body

The CPAD section body contains the information needed to perform the action. Component suppliers may define proprietary section bodies. More details will be provided in a future release of this specification.

8.4.4. Platform Action Event Reporting

The results of platform actions requested via CPADs shall be reported through the Platform Action Status queue using Platform Action Event CPER structures defined in this section. See the [example sequence diagrams](#) in this document to see examples of how RAS API endpoints generate Platform Action Event CPERs when they receive CPADs.

Each CPAD section specifies an action. Every CPAD section shall result in one or more Platform Action Event CPER sections that contain the status of the action. The Platform Action Event CPER sections contain data sufficient to map them to the original CPAD section that specified the action.

The following example specifies a CPAD that comprises three actions and how Platform Action Events might be generated.

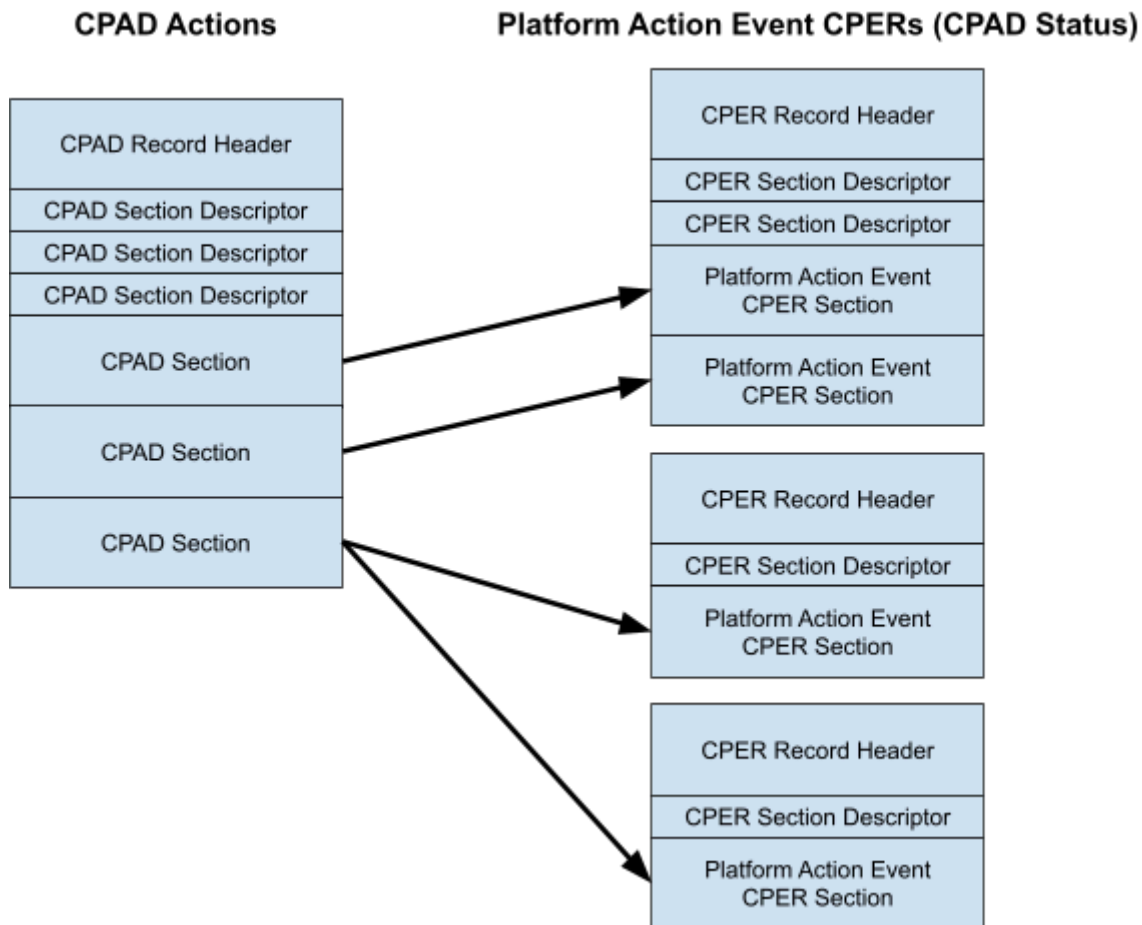


Figure 11: CPAD status information in Platform Action Events

In the example above, a single CPAD containing three sections results in three Platform Action Event CPERs. The third CPAD section results in two Platform Action Events. This might occur when a CPAD action has dependencies, such as a reboot, that are delaying the action. The RAS API endpoint might send a Platform Action Event that indicates that the action is pending and then later send a second Platform Action Event that indicates the final status of the action once the dependencies are met.

Platform Action Events are CPER structures and follow the CPER header format as shown in the table below.

Table 9 – Platform Action Event CPER Header

Mnemonic	Byte Offset	Byte Length	Description
Signature Start	0	4	Same as standard CPER
Revision	4	2	Same as standard CPER
Signature End	6	4	Same as standard CPER
Section Count	10	2	Same as standard CPER
Error Severity	12	4	Same as standard CPER + new: 0 – Recoverable/non-fatal uncorr 1 - Fatal 2 - Corrected 3 - Informational 4 – Platform Action Event (Proposed)
Validation Bits	16	4	Same as standard CPER
Record Length	20	4	Same as standard CPER
Timestamp	24	8	Same as standard CPER
Platform ID	32	16	Same as standard CPER
Partition ID	48	16	Same as standard CPER
Creator ID	64	16	Same as standard CPER
Notification Type	80	16	Same as standard CPER
Record ID	96	8	Same as standard CPER
Flags	104	4	Same as standard CPER
Persistence Information	108	8	Same as standard CPER
Reserved	116	12	Same as standard CPER

Table 10 – Platform Action Event CPER Section Descriptor

Mnemonic	Byte Offset	Byte Length	Description
Section Offset	0	4	Same as standard CPER
Section Length	4	4	Same as standard CPER
Revision	8	2	Same as standard CPER
Validation Bits	10	1	Same as standard CPER
Reserved	11	1	Same as standard CPER
Flags	12	4	Same as standard CPER
Section Type	16	16	Same as standard CPER + new GUID: Platform Action Event Result/Status
FRU Id	32	16	Same as standard CPER
Section Severity	48	4	Same as standard CPER + new 0 – Recoverable/non-fatal uncorr 1 - Fatal 2 - Corrected 3 - Informational 4 – Platform Action Event (Proposed)
FRU Text	52	20	Same as standard CPER

Platform Action Event CPER Section bodies contain status information for a single CPAD section and key data from the CPAD section so that recipients can correlate the status in the Platform Action Event with the action specified in a CPAD section.

Table 11 – Platform Action Event CPER Section Body

The Platform Action Event CPER Type is the following: {0xC7D26A20, 0x0C54, 0x11F1, {0xB3, 0x70, 0x00, 0x15, 0x5D, 0x25, 0xB6, 0x27}}

Mnemonic	Byte Offset	Byte Length	Description
Validation bits	0	2	Bit 0 – Source CPAD Record ID valid Bit 1 – Source CPAD Section Offset valid Bit 2 – Source action ID valid Bit 3 – Action Return code valid Bit 4 – Additional Context valid All other bits are reserved and must be zero

Action Return Code	2	1	Action return codes indicate the status of an action that was received in a CPAD section descriptor. Action return codes are defined in the table below.
Action Return Reason Code	3	1	Action Return Reason Codes provide additional information about the Action Return Code and are defined in tables below.
Source Action ID	4	2	Action ID from source CPAD
Reserved	6	6	
Source CPAD Section Descriptor Index	12	4	Section Descriptor index corresponding to the Action instance in the source CPAD (e.g., 0 for first one, 1 for second one, etc)
Source CPAD Platform ID	16	16	Platform ID value from CPAD that requested the action
Source CPAD Partition ID	32	16	Partition ID value from CPAD that requested the action
Source CPAD Creator ID	48	16	Creator ID value from CPAD that requested the action
Source CPAD Record ID	64	8	Record ID value from CPAD that requested the action
Additional Context	72	X	Optional: This is a variable size field providing additional context information about the results of the action requested. This field is padded with zeros if the size is not a multiple of 16 bytes. The endpoint is expected to include any information necessary for the analyzer to understand the status of the action. This field is specific to CreatorID and their CPAD formats.

The following table lists values defined for the Action Return Code field.

Table 12 – Platform Action Event Action Return Codes

Action Return Code	Meaning
0x00	Action applied successfully
0x01	Action Failed. See Action Return Reason Code for more details.
0x02	The action requested is pending. The endpoint received the action but the action is pending, e.g. the action depends on an external event like a reboot. This status allows analysis tools to see the pending action and may be used to prevent additional CPADs if errors continue to occur prior to the action being taken.
0x03	Policy rejected this CPAD - The server fleet operator rejected this action.
0x04 - 0xFF	Reserved

Table 13 – Platform Action Event Action - Action Applied Successfully Reason Codes

Action Return Reason Code	Meaning for Action Return Code 0x00 (Action applied Successfully)
0x00	No Reason Code
0x80 - 0xFF	Component-supplier-defined - these values are specific to a CreatorID.

Table 14 – Platform Action Event - Action Failed Reason Codes

Action Return Reason Code	Meaning for Action Return Code 0x01 (Action failed)
0x00	No Reason Code
0x01	Error: Unknown error
0x02	Error: EP not ready
0x03	Error: Malformed/invalid CPAD
0x04	Error: Incorrect EP destination
0x05	Error: Invalid FRUID
0x06	Error: Unsupported action
0x07	Error: Action is supported but unconfigured or disabled
0x08	Error: Invalid action data
0x09	Error: Timeout
0x0A	Error: Action dependency failed
0x0B	Error: Action precondition failed
0x80 - 0xFF	Component-supplier-defined - these values are specific to a CreatorID.

Table 15 – Platform Action Event - Action is Pending Reason Codes

Action Return Reason Code	Meaning for Action Return Code 0x02 (Action is pending)
0x00	No Reason Code
0x80 - 0xFF	Component-supplier-defined - these values are specific to a CreatorID.

Table 16 – Platform Action Event - Policy Reject the Action Reason Codes

Action Return Reason Code	Meaning for Action Return Code 0x03 (Policy rejected the action)
0x00	No Reason Code
0x80 - 0xFF	Component-supplier-defined - these values are specific to a CreatorID.

9. Glossary of Terms

Key RAS API terminology is listed in Section-5. For a detailed list for Hardware Fault Management terminology please refer to:

[Link to the OCP Hardware Fault Management Glossary](#)

10. References

1. T. Islam and D. Manivannan, "Predicting Application Failure in Cloud: A Machine Learning Approach," 2017 IEEE International Conference on Cognitive Computing (ICCC), Honolulu, HI, USA, 2017, pp. 24-31, doi: 10.1109/IEEE.ICCC.2017.11.
 2. [OCP Hardware Fault Management Infrastructure Requirements](#)
 3. [OCP Fleet Memory Fault Management Framework Requirements](#)
 4. [UEFI specification - Appendix N](#)
-